Transactional Frameworks for Middleware Platform

Middleware Architecture (SCS 2314)

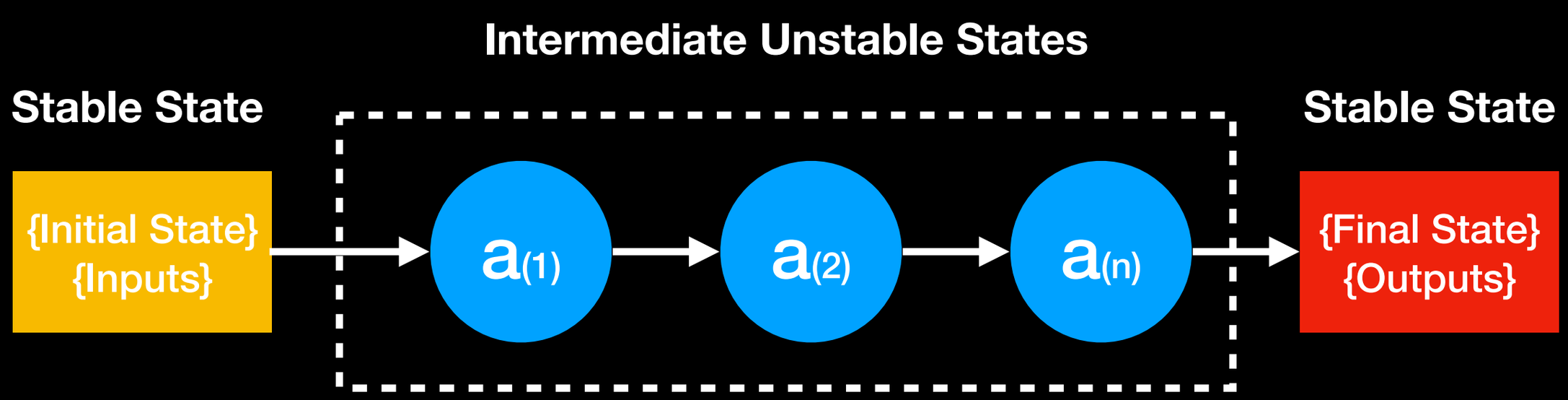
Chaminda Ranasinghe

UCSC

Transaction Atomicity

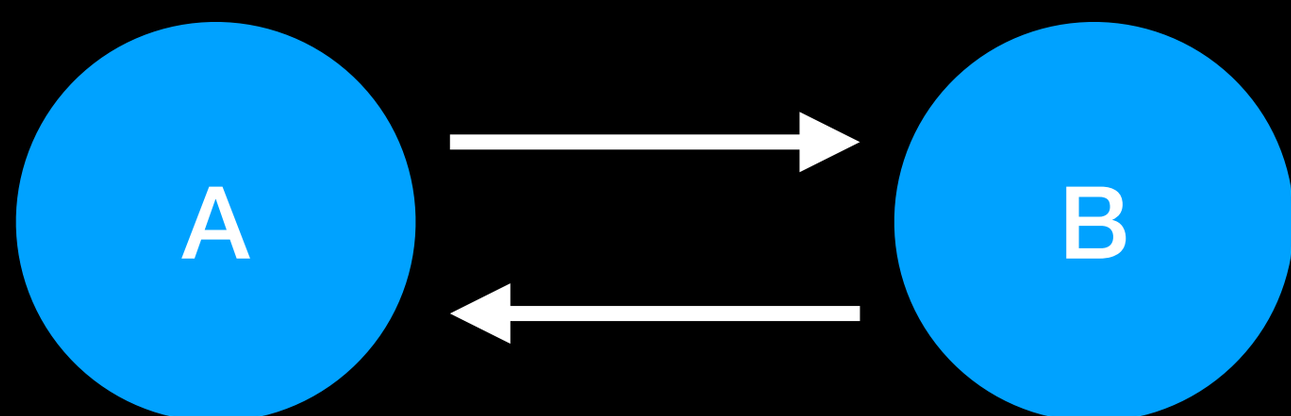
What is a Transaction

- Change from One stable state to another stable state.
- E.g. Banking Domain - Deposit, Funds Transfer



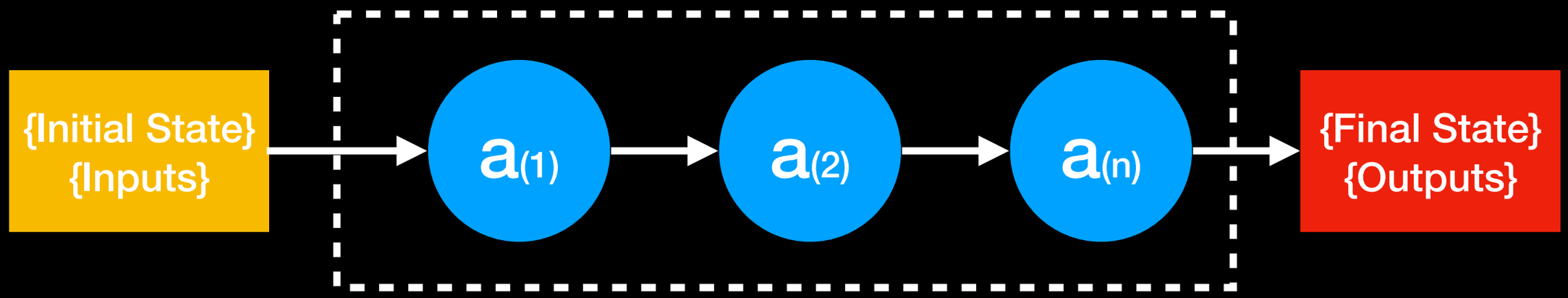
Atomic Action

- The atomic operations (A and B) that are mutually exclusive on dependency can occur without failure in the outcome:
  - When occurring concurrently
  - A occurs first followed by B
  - B occurs first followed by A

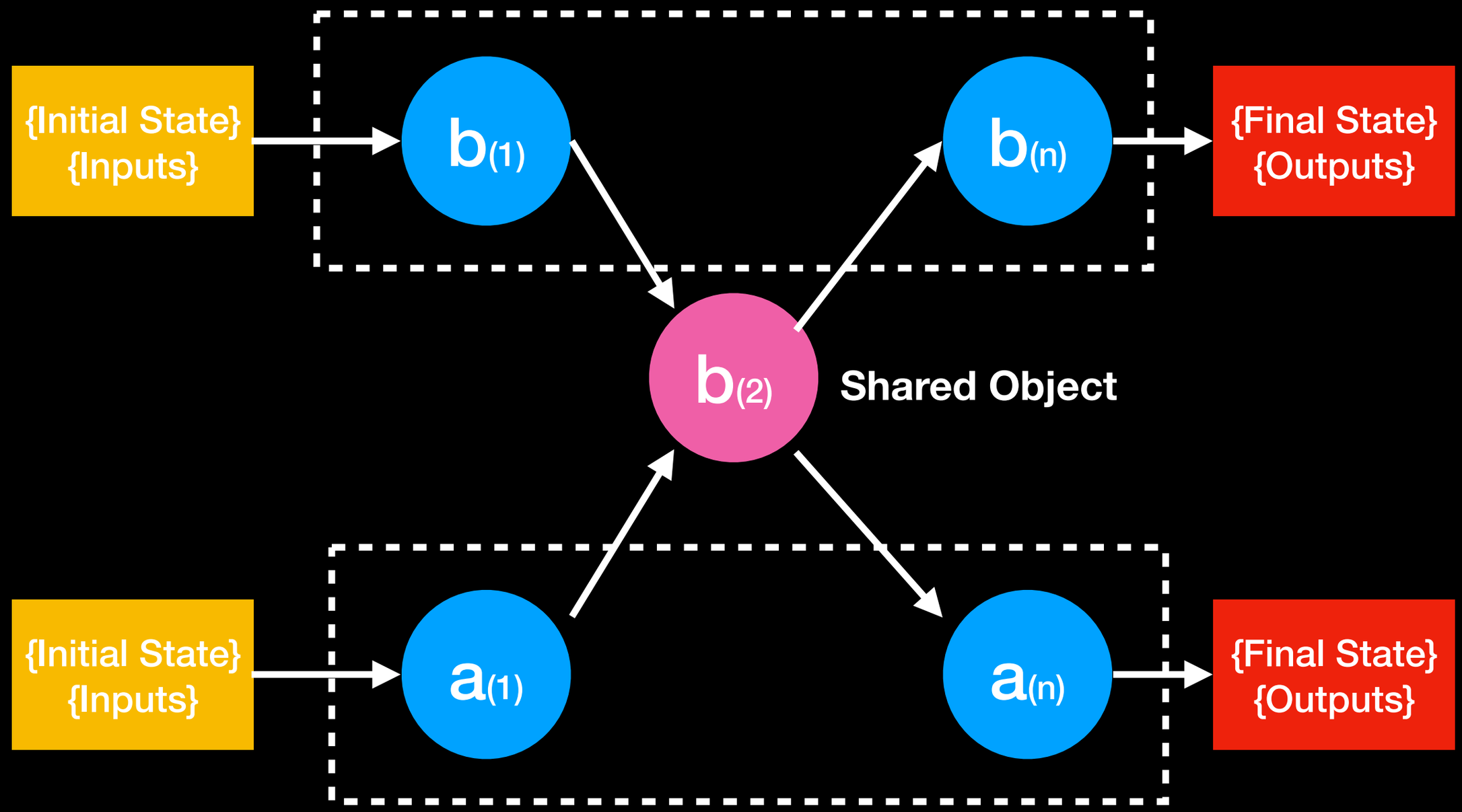


Composite Action - Serialised

- Start with an initial input value and an initial state.
- Reach a final: output value, a final state within a finite time
- Consists of a sequence of atomic operations.



Composite Action - Concurrent



Properties of a Transaction (ACID)

- **A**tomicity - Failure atomicity
- **C**onsistency - Integrity
- **I**solation - Concurrency atomicity
- **D**urability

Atomicity - Failure Atomicity

- A transaction should transfer from one atomic state to another.
- Action A makes a system change from state S1 to S2 - S1 {A} S2.
- Due to failures, an action may decide to interrupt execution and cancel changes, rolling back or aborting.
- The final state would be either S1 or S2
  - S1 - if failure/rollback
  - S2 - if action completed without failure.

Consistency

- A transaction is defined as an atomic or composite action that brings a system from one consistent state to another consistent state.
- During the execution of a transaction, some intermediate states may be inconsistent.
- By atomicity, the inconsistent states are not visible outside the transaction.
- e.g. funds transfer in a banking transaction

Isolation - Concurrency Atomicity

- Concurrent actions by concurrency processes.
- Two actions executed concurrently A || B.
- In the absence of failures, the outcomes are (sequences):
  - A; B or
  - B; A
- Any intermediate states are visible to concurrently executing actions.

Durability

- If a transaction succeeds, the changes done to the state of the system are permanent.
- A successful end of a transaction is called a Commit in databases.
- Durability means that after a transaction has been committed, the changes that it made are immune to failures.

Managing ACID Properties of a Transaction

- Concurrency control through locking
- Transaction finalisation with commit
- Failure handle through rollback/abort

Concurrency Control

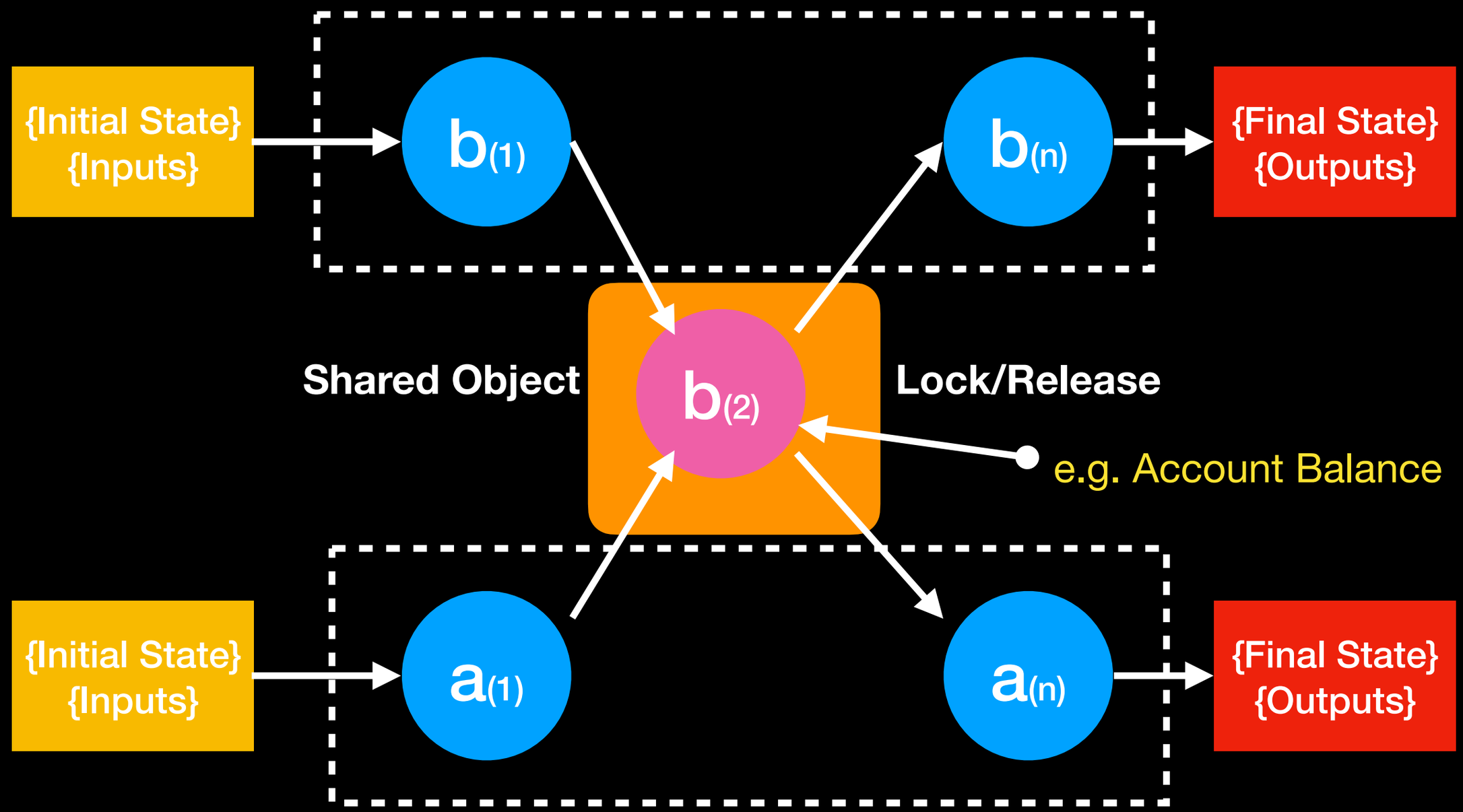
Reasons for Concurrency

- Improve performance of applications - multiple transactions to execute concurrently.
- Make full use of computing resources - CPU, memory, etc

Concurrency Control Basics

- The goal of concurrency control is to ensure isolation if several transactions are allowed to execute concurrently on the same set of data.
- The main concept of concurrency control is serialisability.
- The main mechanism used to enforce serialisability is locking.

Concurrency Control Basics



Serialisability

- A sequence of atomic actions is called a schedule.
- A schedule that is equivalent to a serial schedule is said to be serialisable.

Serialisability Violation

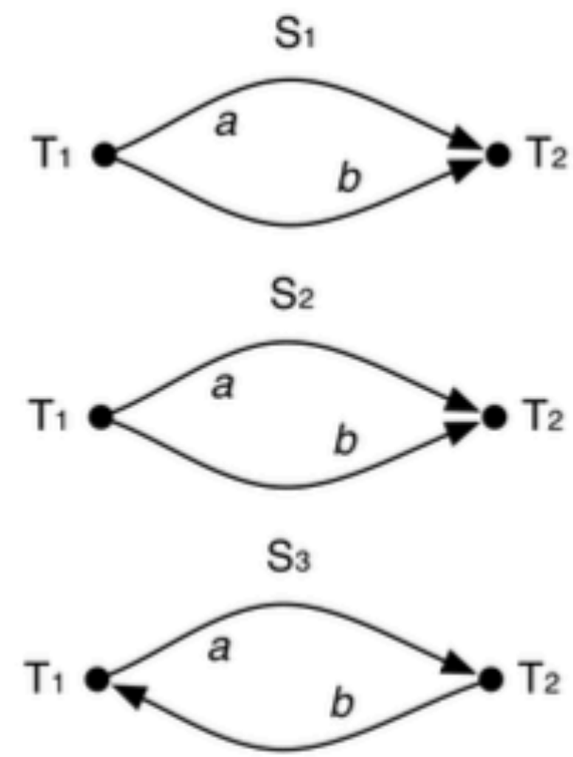
- Transaction - Ti on object x is denoted Ri(x) Wi(x) (Read and Write).
- Consistency may be violated when two transactions Ti and Tj (I != j) execute operations on a shared object x, and at least one of these operations is a write - Such a situation is called a conflict.
- The situations where consistency is violated in Serialisable Schedules:(The shared object is not committed for completion during each of the read or write actions)
  - Unrepeatable read: Ri(x) ...Wj(x) .... Then a subsequent read of x by Ti may deliver a different result from that delivered in a serial execution.
  - Dirty read: Wi(x) ...Rj(x) .... Then the result of Rj(x) may be different from that delivered in a serial execution.
  - Lost write: Wi(x) ...Wj(x) .... Then the effect of the first write may be cancelled by that of the second write.

Consistency Violation Example

Transaction T_1	Transaction T_2
begin	
if $(A \geq X)$	*begin*
$\{A = A - X\}$	$A = (1 + R)A$
else *abort*	$B = (1 + R)B$
$B = B + X$	*commit*
commit	

Consistency Violation Example

Schedule S_1	Schedule S_2	Schedule S_3
$begin_1$	$begin_1$	$begin_1$
$a = a - x$	$a = a - x$	$a = a - x$
$b = b + x$	$begin_2$	$begin_2$
$commit_1$	$a = (1 + r)a$	$a = (1 + r)a$
$begin_2$	$b = b + x$	$b = (1 + r)b$
$a = (1 + r)a$	$commit_1$	$commit_2$
$b = (1 + r)b$	$b = (1 + r)b$	$b = b + x$
$commit_2$	$commit_2$	$commit_1$



S1 is equivalent to S2 and thus is serialisable. S3 has a write-write conflict causing the first write-on b to be lost, violating consistency. The dependency graph of S3 contains a cycle. A schedule is conflict-serialisable if and only if its dependency graph is acyclic.

Locking

Basics of Locking

- To enforce serialisability of concurrent transactions, one needs a mechanism to delay the progress of a transaction.
- Locking is the most frequently used technique.
- To allow concurrent access to a shared object in read-only mode, two types of locks are defined:
  - exclusive
  - shared
- To acquire a lock on object o, the primitives are lock exclusive(o) and lock shared(o).
- An acquired lock is released by unlock(o).

Rules for Locking Shared Objects

- A transaction needs to acquire a lock on a shared object before using it.
- Shared lock for read-only access, exclusive lock for read-write access.
- A transaction may not lock an object on which it already holds a lock, except to upgrade a lock from shared to exclusive.
- No object should remain locked after the transaction concludes, through either commit or abort.

Two Phase Locking (2PL)

- 2PL Rule: After a transaction has released a lock, it may not obtain any additional locks.
- A transaction that follows this rule has two successive phases:
  - Phase 1 - it acquires locks
  - Phase 2 (shrinking) - it releases locks.
- The point preceding the release of the first lock is called the maximum locking point.

2PL Example

Transaction T_{1a}	Transaction T_{1b}	Transaction T_2
$begin_{1a}$	$begin_{1b}$	$begin_2$
$lock_exclusive_{1a}(a)$	$lock_exclusive_{1b}(a)$	$lock_exclusive_2(a)$
$a = a - x$	$a = a - x$	$a = (1 + r)a$
$unlock_{1a}(a)$	$lock_exclusive_{1b}(b)$	$lock_exclusive_2(b)$
$lock_exclusive_{1a}(b)$	$unlock_{1b}(a)$	$b = (1 + r)b$
$b = b + x$	$b = b + x$	$unlock_2(b)$
$unlock_{1a}(b)$	$unlock_{1b}(b)$	$unlock_2(a)$
$commit_{1a}$	$commit_{1b}$	$commit_2$

Which transactions are 2PL compliant? Apply 2PL Rule

2PL Example

Transaction T_{1a}	Transaction T_{1b}	Transaction T_2
$begin_{1a}$	$begin_{1b}$	$begin_2$
$lock_exclusive_{1a}(a)$	$lock_exclusive_{1b}(a)$	$lock_exclusive_2(a)$
$a = a - x$	$a = a - x$	$a = (1 + r)a$
$unlock_{1a}(a)$	$lock_exclusive_{1b}(b)$	$lock_exclusive_2(b)$
$lock_exclusive_{1a}(b)$	$unlock_{1b}(a)$	$b = (1 + r)b$
$b = b + x$	$b = b + x$	$unlock_2(b)$
$unlock_{1a}(b)$	$unlock_{1b}(b)$	$unlock_2(a)$
$commit_{1a}$	$commit_{1b}$	$commit_2$

Acquire locks after unlocking

T1b and T2 are 2PL compliant,

T1a is not 2PL compliant

2PL Example

$S_a(T_{1a}, T_2)$	$S_b(T_{1b}, T_2)$
$begin_{1a}$	$begin_{1b}$
$lock_exclusive_{1a}(a)$	$lock_exclusive_{1b}(a)$
$a = a - x$	$a = a - x$
$unlock_{1a}(a)$	$lock_exclusive_{1b}(b)$
$begin_2$	$unlock_{1b}(a)$
$lock_exclusive_2(a)$	$begin_2$
$a = (1 + r)a$	$lock_exclusive_2(a)$
$lock_exclusive_2(b)$	$a = (1 + r)a$
$b = (1 + r)b$	$b = b + x$
$unlock_2(b)$	$unlock_{1b}(b)$
$lock_exclusive_{1a}(b)$	$lock_exclusive_2(b)$
$b = b + x$	$b = (1 + r)b$
$unlock_2(a)$	$commit_{1b}$
$commit_2$	$unlock_2(b)$
$unlock_{1a}(b)$	$unlock_2(a)$
$commit_{1a}$	$commit_2$

Find where the inconsistency is created?

Two possible schedules Sa and Sb, respectively generated by the concurrent execution of T1a, T2 and T1b, T2.

2PL Example

$S_a(T_{1a}, T_2)$	$S_b(T_{1b}, T_2)$
$begin_{1a}$	$begin_{1b}$
$lock_exclusive_{1a}(a)$	$lock_exclusive_{1b}(a)$
$a = a - x$	$a = a - x$
$unlock_{1a}(a)$	$lock_exclusive_{1b}(b)$
$begin_2$	$unlock_{1b}(a)$
$lock_exclusive_2(a)$	$begin_2$
$a = (1 + r)a$	$lock_exclusive_2(a)$
$lock_exclusive_2(b)$	$a = (1 + r)a$
$b = (1 + r)b$	$b = b + x$
$unlock_2(b)$	$unlock_{1b}(b)$
$lock_exclusive_{1a}(b)$	$lock_exclusive_2(b)$
$b = b + x$	$b = (1 + r)b$
$unlock_2(a)$	$commit_{1b}$
$commit_2$	$unlock_2(b)$
$unlock_{1a}(b)$	$unlock_2(a)$
$commit_{1a}$	$commit_2$

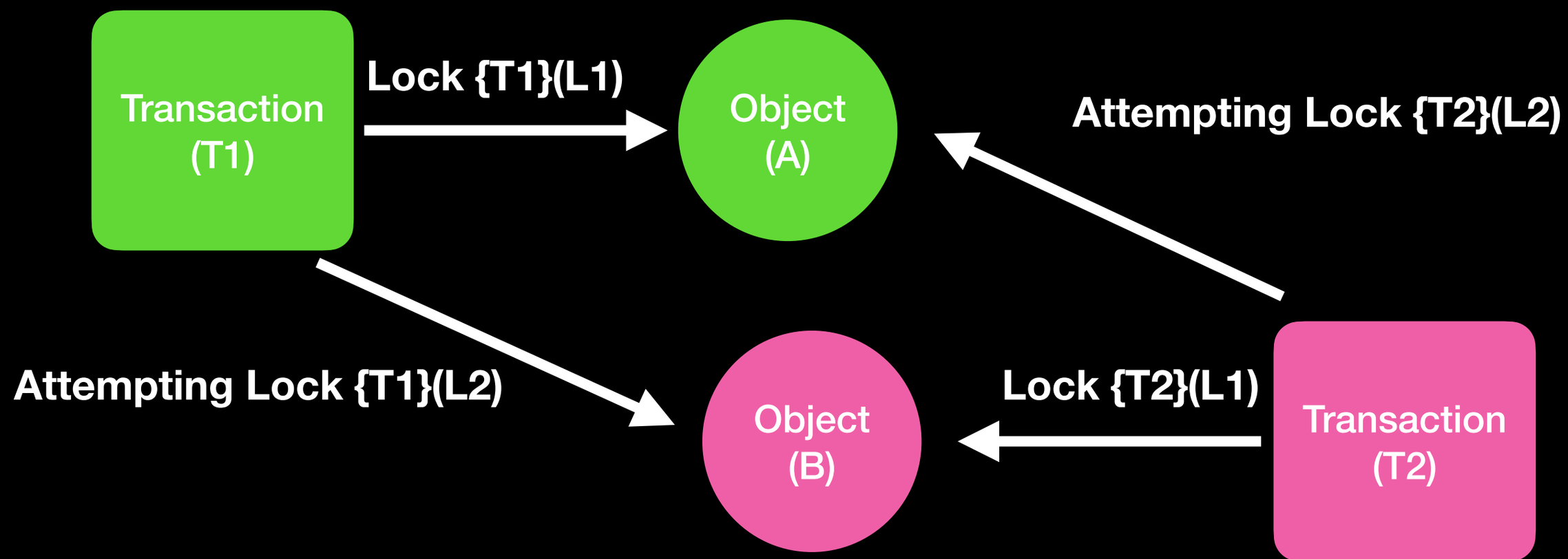
Schedule Sa violates consistency since T2 operates on an inconsistent value of b.

Schedule Sb is equivalent to the serial schedule T1b + T2

Avoiding Deadlocks

Deadlock Avoidance

- Using locks raises the risk of deadlock, a situation of circular wait for shared resources.



Avoidance of Deadlocks - Prevention

- A common prevention method is that all transactions should lock the objects in the same locking order.
- However, this is not always feasible, since in many applications transactions are independent and do not have an advanced knowledge of the objects they need to lock.
- Imposing a locking order may be overly restrictive, and may thus reduce concurrency.

Avoidance of Deadlocks - Detection & Resolution

- Detection involves two design decisions:
  - Defining a deadlock detection algorithm
  - Defining a deadlock resolution method
- Deadlock detection amounts to finding a cycle in the wait-for graph.
- In a distributed system, this involves information interchange between the nodes, so that the deadlock may be detected locally at any site.
- Once detected, a deadlock is resolved by aborting one or more "victim" transactions.
- A victim is chosen to minimise the cost of its rollback according to some criterion (e.g., choosing the "youngest" transaction, or choosing a transaction local to the detection site, if such a transaction exists).

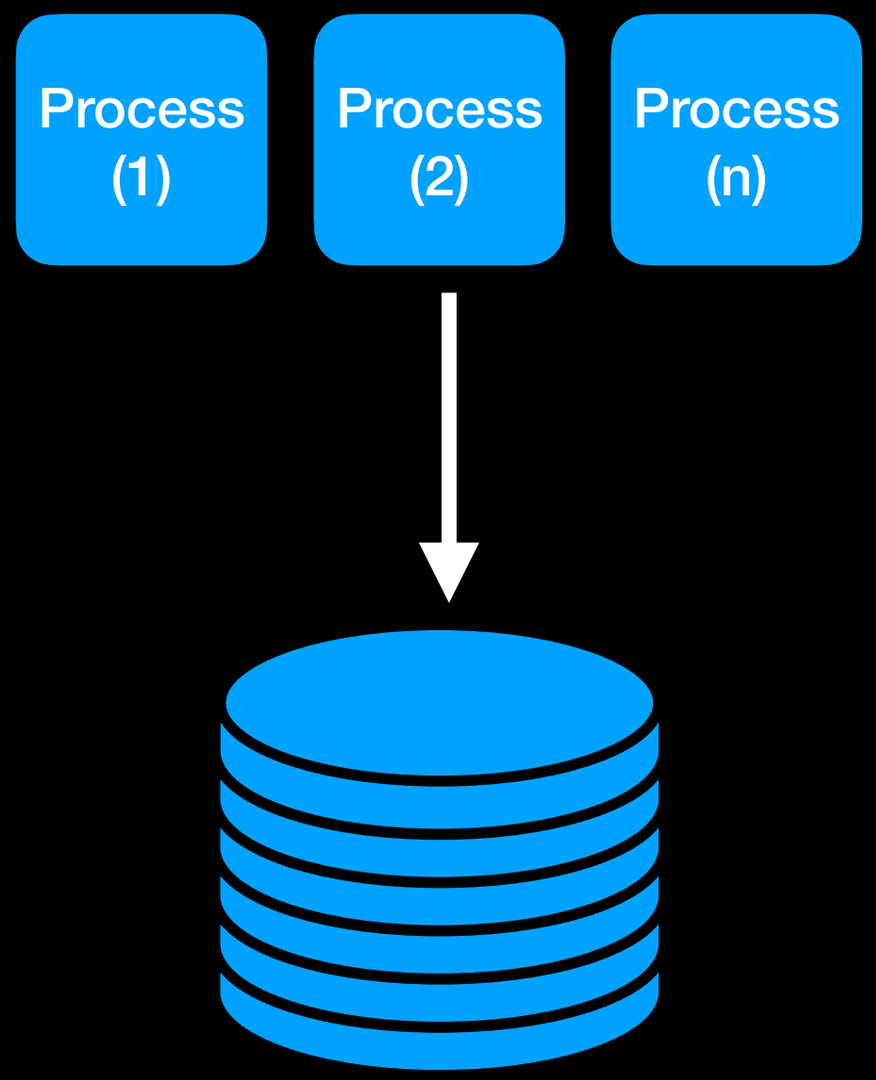
Transactional Middleware

Transactional Middleware

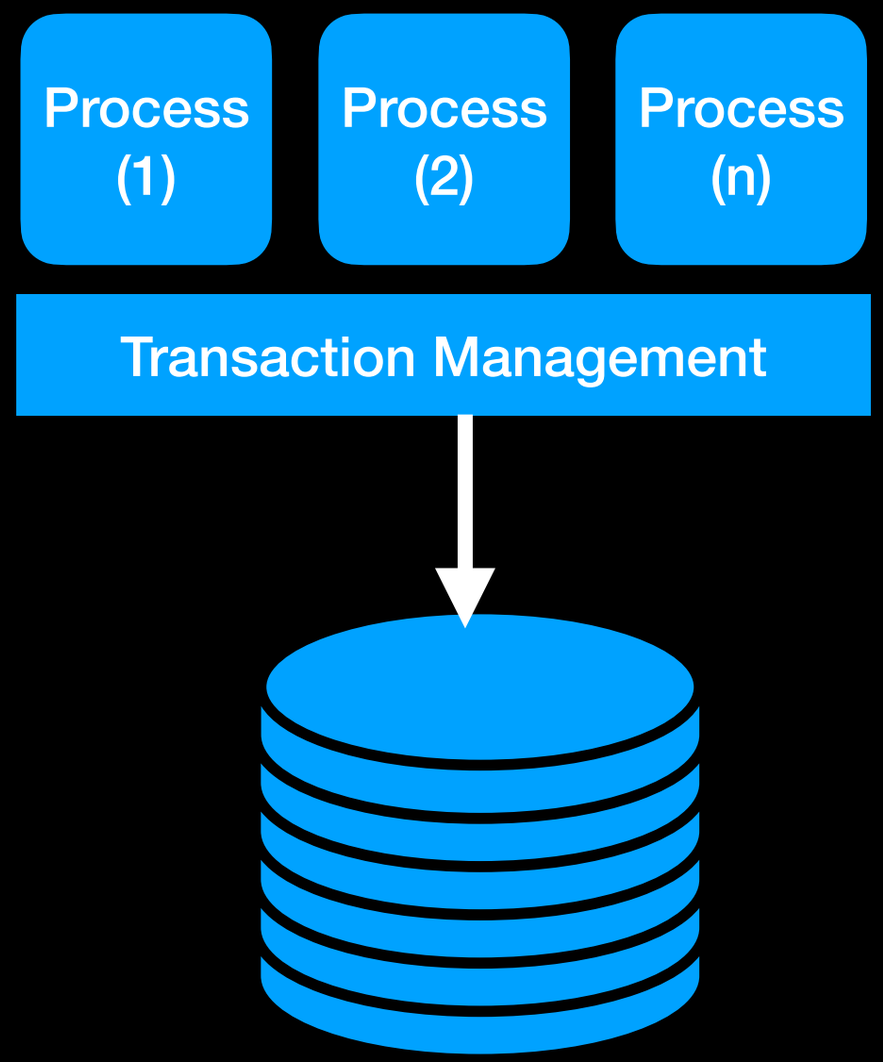
- In centralised databases, transactions were supported by a set of modules closely integrated within database management systems.
- As systems became distributed, and as transactions were applied to non-database services such as messaging, transaction management was isolated within specialised middleware components.
- A transaction processing system is typically organised into a transaction manager (TM) and a set of resource managers (RM).
- Each resource manager is in charge of a local resource, such as a database or a message queuing broker, and is responsible for ensuring transactional properties for access to the local resource.

Transactional Middleware

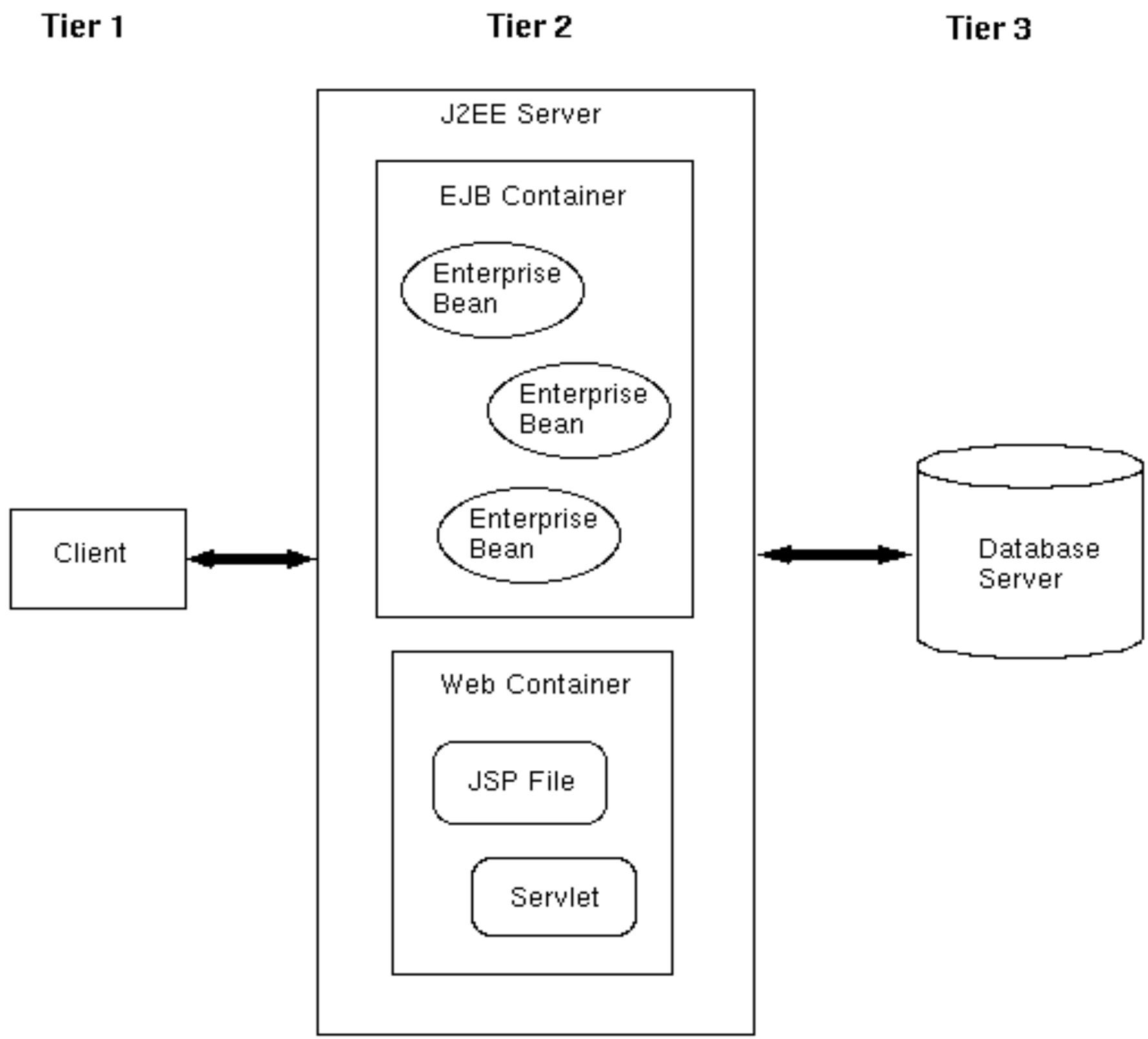
Initially applications assigned the transaction management to the Persistence Tier (DBMS)



With distributed concurrent processing, transaction management could be assigned to Business Logic Tier



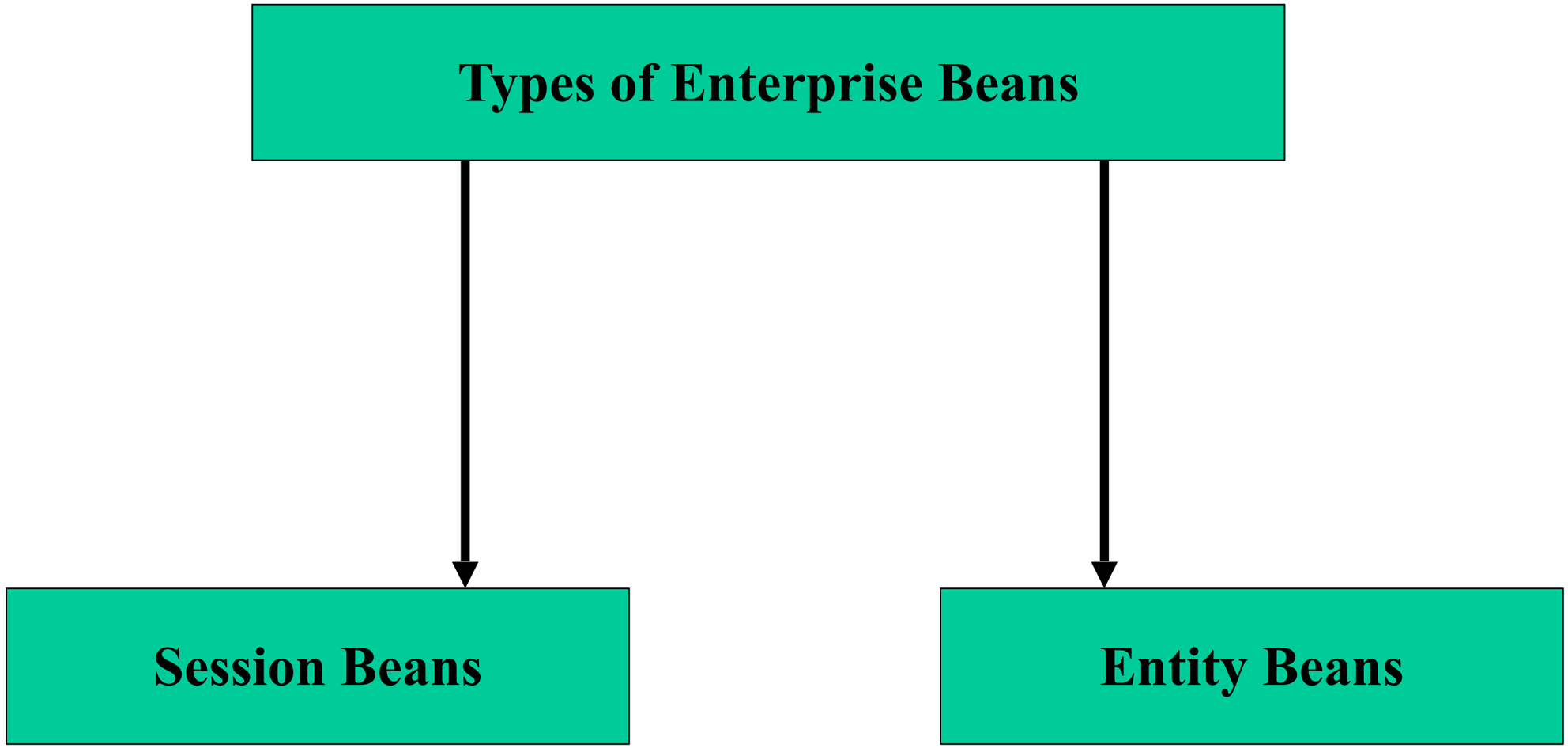
JEE Architecture



JEE Architecture - EJB Container

- Enterprise bean instances run within an EJB container.
- The container is a runtime environment that controls the enterprise beans and provides them with important system-level services.
- Services provided by Enterprise Beans.
  - Transaction Management
  - Security
  - Remote Client Connectivity
  - Life Cycle Management
  - Database Connection Pooling

Types of EJBs

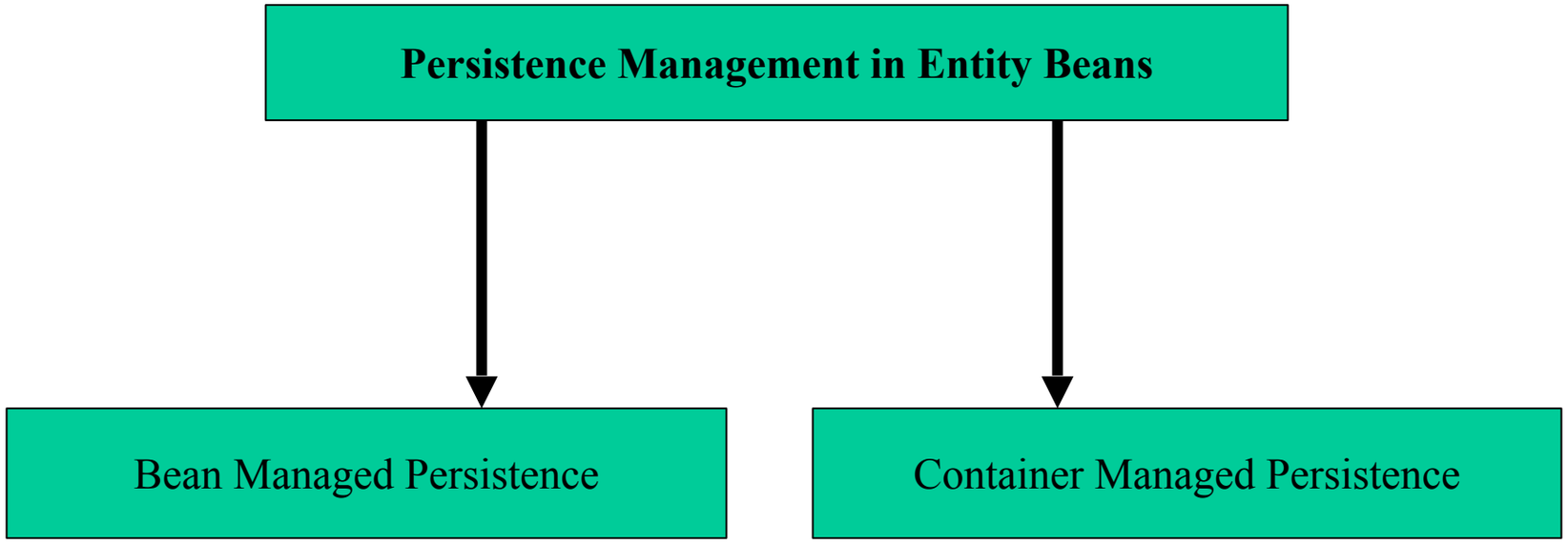


Session Beans

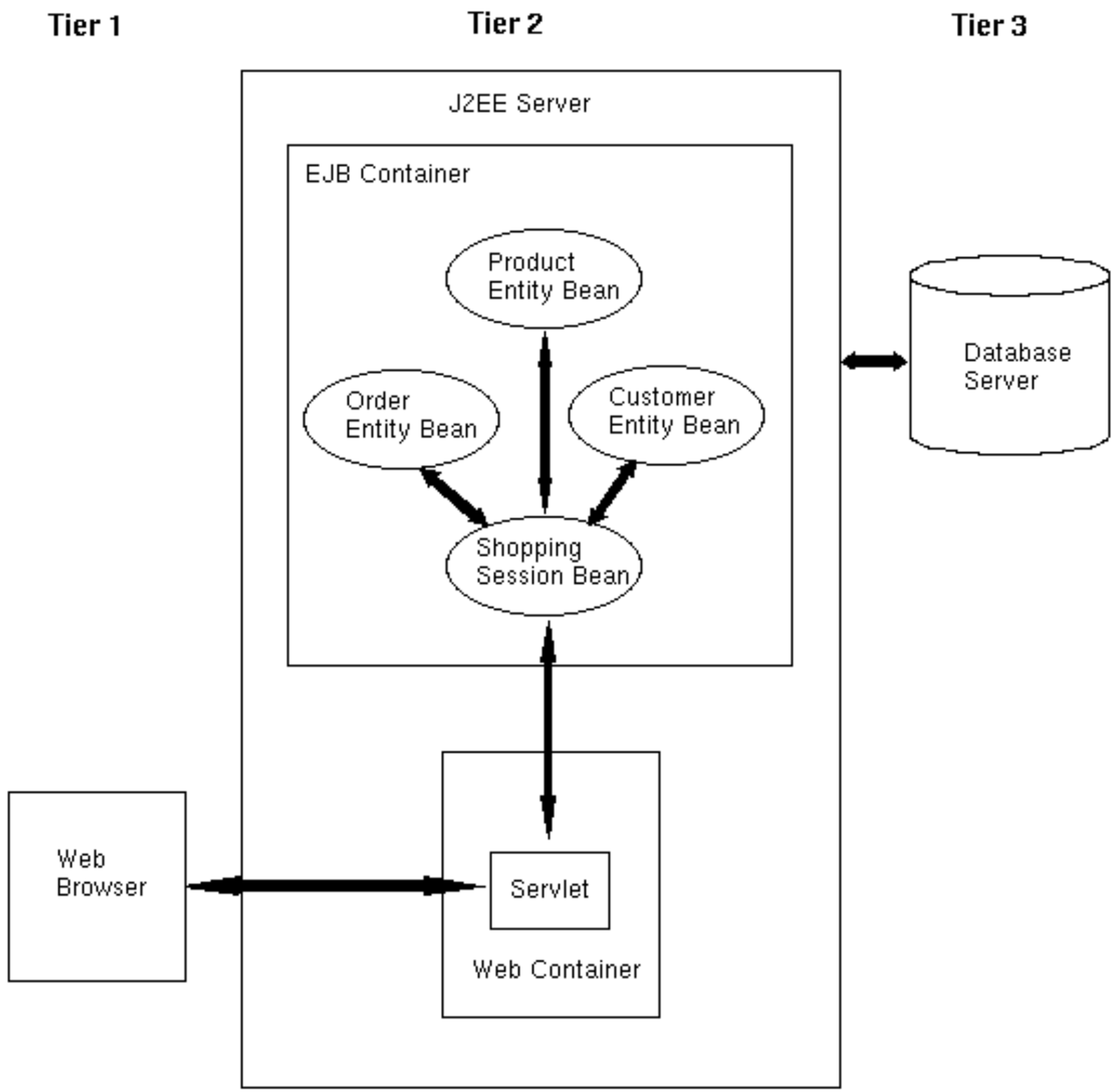
- A session bean represents a client in the J2EE server (Facade Pattern).
- A client communicates with the J2EE server by invoking the methods that belong to an enterprise.
- A session bean converses with the client and can be thought of as an extension of the client.
- Each session bean can have only one client.
- When the client terminates, its corresponding session bean also terminates. Therefore, a session bean is transient, or non-persistent.

Entity Beans

- An entity bean represents a business object (Subject Domain Object) in a persistent storage mechanism such as a database.
- An entity bean's information does not have to be stored in a relational database.
- It could be stored in an object database, a legacy application, a file, or some other storage mechanism.



EJB Application Example



{ e n d }

An interesting thing about touch is that the brain doesn’t just tell you how something feels, but how it ought to feel. That’s why the caress of a lover feels wonderful, but the same touch by a stranger would feel creepy. It’s also why it is so hard to tickle yourself.

The Body: A Guide for Occupants

Chaminda Ranasinghe

UCSC